



TXV1_OBD_Dongle_Firmware_Functional_ Specification

Document Number: TRAXO/M6/FW/SB-SPEC/Rev 1.0

Version: 0.3

Date: 08/12/2025

Prepared By: Traxo Engineering Team

I. REVISION HISTORY

Rev. No	Approved Date	Affected/ Sections Pages	Description
0.1	03/12/2025	All	Initial template release
0.2	05/12/2025	Sec 3	Addressed peer review comments
0.3	07/12/2025	Sec 4.4	To be Addressed
1.0	09/12/-2025		

II. TABLE OF CONTENTS

I. REVISION HISTORY.....	2
1. INTRODUCTION.....	4
1.1 Purpose.....	4
1.2 Scope.....	4
1.3 Definitions, Acronyms & Abbreviations.....	5
1.5 Document Structure	5
2. SYSTEM OVERVIEW.....	6
2.1 Product Summary.....	6
2.2 Firmware Responsibilities.....	6
2.3 System Context.....	7
2.4 Operating Conditions	7
2.5 Assumptions	8
3. DEVICE LIFECYCLE MANAGEMENT	9
3.1 Overview	9
3.2 State Definitions.....	9
3.3 State Transition Logic.....	14
3.4 Identity Usage Model	14
3.5 API Interactions Per State.....	14
3.6 Error / Exceptional Conditions.....	14
4. FIRMWARE FUNCTIONAL REQUIREMENTS.....	16
4.1 OBD-II / CAN Signal Handling.....	16
4.2 LED Indication System	22
4.3 Telemetry Publishing Logic.....	25
4.4 Event Framework.....	28
4.5 Alert Framework.....	29
4.6 Trip Management	35
5. Southbound Communication Requirements	41
6. Annexures	44
Annexure A — CAN Signal Definitions	44
Annexure B — Trip State Machine Diagram.....	46
Annexure C — Configuration Parameters and Thresholds	47
7. References.....	50

1. INTRODUCTION

1.1 Purpose

The purpose of this document is to define the firmware functional behaviour of the Neoway-based OBD Dongle used in the vehicle telematics ecosystem. This specification describes how the firmware interacts with the vehicle CAN/OBD-II interface, internal subsystems, LED indicators, and the CVIP cloud platform through southbound communication protocols.

It establishes the required firmware behaviour for device lifecycle management, VIN decoding, telemetry publishing, event and alert handling, trip computation, and cloud communication, ensuring a consistent and verifiable implementation across firmware releases.

1.2 Scope

This specification defines the behaviour of the firmware running inside the Neoway OBD telematics device.

The scope includes:

- Device lifecycle handling
- CAN/OBD-II signal processing
- LED indication logic
- Telemetry transmission rules
- Event and alert detection
- Trip computation logic
- Southbound API interactions with the CVIP cloud platform

Out of scope for this document:

- Hardware schematics and electrical design
- Mobile application logic
- Cloud backend implementation
- Detailed FOTA protocol specifications (only minimal behavioural references may be included)

1.3 Definitions, Acronyms & Abbreviations

Term	Definition
OBD-II	On-Board Diagnostics Interface (automotive standard)
CAN	Controller Area Network
VIN	Vehicle Identification Number
IMEI	International Mobile Equipment Identity
MSISDN	Mobile Station International Subscriber Directory Number
CVIP	Cloud Vehicle Integration Platform
TBOX	Telematics Box (OBD Dongle)
MIL	Malfunction Indicator Lamp
IGN	Ignition state derived from CmdIgnSts CAN signal
Telemetry	Periodic or conditional data sent to cloud
Event	Change-based state notification
Alert	High-priority condition requiring notification
FSM	Finite State Machine
BH2-CAN	OEM-specific CAN bus used for signal acquisition

1.5 Document Structure

- Introduction: Purpose, scope, terminology, and references.
- System Overview: Firmware responsibilities and system interactions.
- Device Lifecycle Management: State definitions, transitions, and cloud interactions.
- Firmware Functional Requirements: CAN processing, LED logic, telemetry, events, alerts, and trip computation.
- Southbound Cloud Interaction Requirements: API mapping, telemetry/events/alerts upload, and identity usage.

2. SYSTEM OVERVIEW

2.1 Product Summary

The Neoway OBD Dongle is a vehicle-mounted telematics device that interfaces with the vehicle's CAN/OBD-II network and communicates vehicle parameters to the CVIP cloud backend.

The firmware is responsible for acquiring CAN signals, decoding essential vehicle information, computing trip and event states, and transmitting telemetry at defined intervals based on ignition status.

It also manages internal lifecycle states to ensure secure onboarding, provisioning, and customer activation.

2.2 Firmware Responsibilities

The firmware implements the following high-level responsibilities:

- Device Lifecycle Control
 - Manage transitions through Pre-Sales, Factory, Provisioned, Authorized, and Customer states.
 - Interact with cloud APIs for certificate download, state transitions, and provisioning.
- CAN/OBD-II Signal Processing
 - Decode VIN from CAN ID 0x3E0.
 - Detect engine state from CmdIgnSts (0x46C).
 - Monitor MIL status via ENGINE_MIL_STAT (0x763).
- Telemetry Publishing
 - Publish periodic telemetry every 60 seconds while engine is ON.
 - Publish two additional indicator telemetry samples when the engine transitions to OFF.
- Event & Alert Handling
 - Detect ignition transitions and connection-status events.
 - Generate MIL alerts and device-specific driving alerts (towing, harsh events, etc., where applicable).

1. Trip Computation

- Compute TRIP_IDLE, TRIP_PENDING, and TRIP_ACTIVE states.
- Apply user-configured thresholds such as MIN_TRIP_DISTANCE and MIN_IGN_OFF_TIME.

2. LED Indicator Management

- Control GPS, GSM, CAN, and Power LED patterns based on subsystem status.

3. Southbound Communication

- Transmit telemetry, events, alerts, and state updates to the CVIP backend.

2.3 System Context

The firmware operates within the following system context:

Vehicle Side

- Receives CAN frames from the BH2-CAN network and decodes VIN, ignition state, MIL status, and other applicable vehicle signals.

Device Internal Subsystems

The firmware interfaces with:

- GNSS module for GPS lock status
- Cellular modem for GSM connectivity
- LED indicators for subsystem health signalling
- Accelerometer for motion-based alerts (if supported by hardware)

Cloud Side (CVIP)

The device communicates with the CVIP backend for:

- Device state reporting
- Telemetry uploads
- Event and alert notifications
- Provisioning and authorization flows

2.4 Operating Conditions

The firmware is expected to operate reliably under the following conditions:

- Power Source: Vehicle battery through OBD-II connector
- Voltage Variations: Engine cranking and start-stop cycles
- Environment: Typical automotive temperature and vibration ranges
- CAN Availability: CAN signals may be unavailable during certain ignition states; firmware must handle such cases gracefully

- Network Availability: GSM network may be intermittent; firmware must retry transmissions as applicable

2.5 Assumptions

- The CVIP backend is available for provisioning, state updates, and telemetry ingestion.
- The Neoway module provides a stable AT command interface for GSM communication and data upload.
- GNSS functionality depends on antenna exposure and satellite visibility.
- Required trip and alert configuration parameters are available to the firmware at runtime.

3. DEVICE LIFECYCLE MANAGEMENT

3.1 Overview

The OBD Dongle firmware operates within a structured lifecycle model that ensures device security, controlled activation, and predictable operational behaviour across different deployment phases.

This lifecycle consists of five states: Factory, Provisioned, Authorized, and Customer, each representing a clearly defined stage in the device's journey from manufacturing to full telematics operation.

The lifecycle model guarantees that the device:

- binds itself to the correct vehicle through VIN acquisition,
- is authenticated and validated by the backend prior to operational use, and
- enables customer-facing telemetry only after explicit activation.

Each state imposes strict behavioural constraints on the firmware regarding what functions may or may not be performed, what identity must be used for communication, which subsystems must be active, and how transitions should occur.

This section describes each lifecycle state in detail, along with operational characteristics, cloud interactions, and the expected transition logic.

3.2 State Definitions

3.2.1 Pre-Sales (Post Firmware Flash)

Purpose

The Pre-Sales state represents the initial firmware state immediately after production firmware flashing and end-of-line verification.

In this phase, the device has not yet interacted with a vehicle or backend system in any meaningful way, and therefore performs only foundational security and network procedures necessary to prepare it for provisioning.

This state ensures that the device enters the field with its cryptographic identity uninitialized, preventing premature data generation or unintended backend registration.

Operational Characteristics

- IMEI is the only identity used. The device has no association with a VIN or customer, and all cloud communication is expected to rely solely on IMEI.
- Vehicle-facing interfaces remain inactive. CAN monitoring, OBD decoding, telemetry processing, event generation, and trip logic are fully disabled.

- Radio subsystems may operate in minimal configuration. The GSM module may only be activated to obtain the access token for CSR; GPS and CAN subsystems remain idle.
- LED indicators may show only basic device readiness. No telematics-related LED patterns should be visible at this stage.

Mandatory Firmware Behaviour

- Initiate CSR access-token request using IMEI as the identity.
- Establish GSM connectivity sufficient to communicate with CVIP for provisioning purposes.
- Persist the Pre-Sales lifecycle state in non-volatile memory to ensure correct state restoration after reboot.
- Do not attempt any form of data publication, vehicle data monitoring, or customer-level feature activation.

Cloud Interaction

API Used:

/dongle/{IMEI}/{protocol}{dataformat}/csrAccessTokenReq

Exit Condition

The device exits the Pre-Sales state when it detects active CAN, GPS, and GSM signals indicating installation in a vehicle environment and readiness for VIN acquisition.

3.2.2 Factory

Purpose

The Factory state is designed specifically for **vehicle association**.

Here, the firmware must identify the vehicle by extracting a valid VIN from CAN traffic. This state ensures that each device is bound to a single, correct vehicle and prevents the possibility of misassociation.

Operational Characteristics

- IMEI continues to be the identity used. VIN is not yet accepted by the backend for device identification.
- CAN subsystem actively listens for VIN on ID 0x3E0. The firmware reconstructs VIN using a sequence of VIN_MSG and VIN_DATA frames.
- GNSS and GSM subsystems remain active. Network and positioning capabilities are ready but no operational data is sent.

- Telemetry and telematics features remain disabled. No data should be transmitted to the backend during this phase.

Mandatory Firmware Behaviour

- Begin VIN acquisition immediately upon detecting active CAN traffic.
- Assemble the VIN from sequence-numbered CAN frames and validate its integrity.
- Persist the VIN in secure non-volatile storage.
- Maintain full readiness of radio and GPS subsystems without enabling operational behaviours.
- Continue VIN scanning until a valid 17-character VIN is extracted.

Cloud Interaction

- No mandatory API call.
The focus of this state is VIN discovery and local preparation for provisioning.

Exit Condition

Transition occurs when the VIN is successfully decoded and the firmware is ready to initiate certificate provisioning.

3.2.3 Provisioned

Purpose

The Provisioned state indicates that the device has successfully completed initial provisioning and received a signed operational certificate.

At this stage, the device is formally introduced to the CVIP backend as a secure, VIN-associated entity.

Operational Characteristics

- Identity transitions to VIN. All backend communication now uses the VIN as the device identifier.
- Certificate-based authentication becomes active. The device now holds its operational certificate for secure cloud communication.
- Telematics features remain disabled. Telemetry, alerts, events, and trip logic remain inactive until authorization.
- Subsystems remain monitored but idle. CAN/GPS/GSM are active but do not trigger data transmission.

Mandatory Firmware Behaviour

- Download the device certificate from CVIP and store it securely.
- Announce completion of provisioning using the deviceJoined message.
- Maintain active monitoring of network, GPS, and CAN status while avoiding operational data handling.
- Persist the Provisioned lifecycle state to handle power cycles safely.

Cloud Interaction

- **API Used:**

/dongle/{VIN}/{protocol}{dataformat}/deviceJoined

Exit Condition

Transition occurs when the device receives an explicit authorization command from CVIP.

3.2.4 Authorized

Purpose

The Authorized state indicates backend authentication and validation after provisioning. CVIP now recognizes the device as a legitimate VIN-linked telematics entity, but operational telematics transmission must not begin until customer activation is completed.

Operational Characteristics

- Device is validated but not customer-active.
- Identity remains VIN.
- Telemetry and alerts remain disabled.
- LED patterns may reflect subsystem readiness without implying operational activation.

Mandatory Firmware Behaviour

- Store lifecycle state as AUTHORIZED in persistent memory.
- Listen for customer activation commands (cloud or SMS).
- Keep all subsystems active and validated (GSM, GPS, CAN).
- Suppress all telemetry and operational triggers.

Cloud Interaction

- API Used:

/dongle/vinnumber/{protocol}{dataformat}/commandResponse

Exit Condition

Transition occurs when CVIP or SMS-based logic sends customer activation command.

3.2.5 Customer

Purpose

The Customer state is the final operational state in which all telematics functions are enabled.

The device is now assigned to a specific end user, and all data capture, processing, and communication workflows are active.

Operational Characteristics

- Identity transitions to MSISDN. Cloud routing is now subscriber-based.
- Telemetry fully active. The device publishes telemetry every 60 seconds during engine ON and generates D5/D6 samples after engine OFF.
- Events and alerts are enabled. Ignition transitions, MIL status, towing, harsh driving, and connection status are fully monitored.
- Trip FSM operates continuously. Idle to Pending to Active transitions occur based on engine status and movement thresholds.

Mandatory Firmware Behaviour

- Enable all telematics modules including telemetry, events, alerts, and trip logic.
- Maintain full monitoring of GPS, GSM, and CAN subsystems.
- Persist Customer state across all resets.
- Respond to backend command messages using MSISDN identity.

Cloud Interaction

API Used:

/dongle/vinnumber/{protocol}{dataformat}/commandResponse

Exit Condition

Customer is the terminal operational state; backward transitions are not permitted.

3.3 State Transition Logic

Pre-Sales → Factory → Provisioned → Authorized → Customer

Transition Rules

- **Pre-Sales to Factory:** Occurs when active CAN, GPS, and GSM signals indicate vehicle installation.
- **Factory to Provisioned:** Occurs after successful VIN decoding and readiness for certificate provisioning.
- **Provisioned to Authorized:** Triggered by authorization command from CVIP.
- **Authorized to Customer:** Triggered when customer registration is complete and CVIP issues activation command.

3.4 Identity Usage Model

Table 3.1 — Device Identity Usage Across Lifecycle States

State	Identity Used	Purpose
Pre-Sales	IMEI	Manufacturing identification and CSR token request
Factory	IMEI	VIN discovery and pre-provisioning
Provisioned	VIN	Secure, certificate-based identity
Authorized	VIN	Backend-validated identity awaiting activation
Customer	MSISDN	SIM-linked operational identity

3.5 API Interactions Per State

Table 3.2 — API Endpoints Mapped to Device Lifecycle States

State	API Endpoint	Description
Pre-Sales	/dongle/{IMEI}/.../csrAccessTokenReq	CSR token acquisition
Factory	None	VIN acquisition only
Provisioned	/dongle/{VIN}/.../deviceJoined	Cloud join message
Authorized	/dongle/vinnumber/.../commandResponse	Authorization & activation control
Customer	/dongle/vinnumber/.../commandResponse	Operational control messages

3.6 Error / Exceptional Conditions

- VIN not decoded: Device must remain in Factory state and retry indefinitely.
- CSR token failure: Firmware must retry with exponential backoff.

- Certificate download failure: Device remains in Provisioned state until successful.
- Authorization timeout: Device must remain in Provisioned and re-request authorization only when prompted.
- Identity mismatch in commands: Firmware must ignore the command and log the occurrence internally.

4. FIRMWARE FUNCTIONAL REQUIREMENTS

4.1 OBD-II / CAN Signal Handling

The firmware shall continuously interface with the vehicle's CAN bus to acquire, decode, and validate essential OBD-II signals required for VIN identification, ignition state tracking, MIL monitoring, telemetry generation, event detection, and trip state computation. The CAN subsystem is a critical source of truth for the dongle; therefore, decoding accuracy, timing responsiveness, and error resilience are mandatory requirements. The following subsections detail all CAN-related behaviours implemented by the firmware.

4.1.1 VIN Decoding

Purpose

VIN decoding serves as the foundation of the device-vehicle binding process.

It is the very first critical step through which the dongle establishes its identity relative to the vehicle in which it is installed.

A correct VIN must be extracted before the device proceeds with cloud provisioning, ensuring that backend systems can uniquely associate the dongle to a specific vehicle and that all subsequent telematics data is attributed to the correct asset. Therefore, the decoding logic must be robust enough to handle fragmented transmissions, intermittent CAN reception, and ordering challenges, while still ensuring a clean, validated, and complete 17-character VIN before lifecycle progression is allowed.

The device must not advance to provisioning or certificate download until the VIN is fully and correctly decoded, verified, and securely stored, as this value forms the permanent binding between the telematics hardware and the vehicle's backend digital identity.

Signal Processing Requirements

1. Continuous Monitoring

The firmware must listen for CAN ID **0x3E0** at all times whenever CAN communication is active.

The listener may not be rate-limited or triggered; it must run continuously to avoid missing any VIN fragments.

2. Fragmented Data Handling

The VIN is transmitted across multiple CAN frames.

- **VIN_MSG** indicates the positional index of the fragment.
- **VIN_DATA** contains 7 bytes of the VIN.

The firmware must reconstruct the VIN strictly in ascending VIN_MSG order, even if frames arrive out of sequence.

3. Assembly Rules

- Concatenate VIN_DATA according to VIN_MSG index positions.
- Discard any overlapping or duplicate VIN_MSG fragments unless confirmed identical.
- Buffer must always track which segments are received and which are pending.

4. Validation Rules

The VIN must be validated against:

- Allowed alphanumeric character set
- Total length = 17 bytes
- Absence of incomplete or missing fragments

5. Persistence Requirements

Once reconstructed, the VIN must be stored in secure, non-volatile memory so that reboots do not require reacquisition unless CAN inconsistencies are detected.

4.1.2 Engine State Handling (Ignition Detection)

Purpose

Ignition state serves as one of the most critical inputs into the telematics firmware, acting as the primary trigger for controlling telemetry scheduling, managing the Trip Finite State Machine (FSM), determining alert evaluation windows, and coordinating multiple timing-dependent behaviours within the device.

Because ignition status directly reflects whether the engine is running or stopped, it influences not only real-time telemetry publication but also trip start/stop boundaries, MIL evaluation timing, and power management strategies.

Incorrect ignition interpretation could result in premature trip starts, missed trip ends, incorrect telemetry patterns, or misaligned alert evaluation, all of which impact the correctness of backend analytics.

The firmware must therefore handle ignition decoding with the same level of priority and precision as VIN and MIL processing.

Signal Interpretation Requirements

The firmware shall derive engine state exclusively from the CmdIgnSts signal received on CAN ID 0x46C from the BH2-CAN network. No alternative signals or fallback heuristics may be used to infer ignition.

The mapping is:

CmdIgnSts	Interpretation	Engine State
0x4	RUN	Engine ON
0x5	START	Engine ON
Other values	ACC / OFF / Unknown	Engine OFF

Mandatory Firmware Behaviour

1. Latency Requirements

The firmware must update its internal ignition state within a single CAN cycle of receiving a new CmdIgnSts frame.

This ensures instantaneous reaction to engine transitions, allowing the device to:

- initiate telemetry scheduling without delay,
- trigger Trip FSM transitions at the correct boundaries,
- align MIL evaluation timing with actual engine runtime, and

2. Event Triggering

Ignition ON and OFF transitions must generate events only when a genuine state change occurs, meaning:

- If CmdIgnSts repeats the same value consecutively, no additional events must be emitted.
- Events must reflect true transitions, not repetitive CAN frames.

Example:

- 0x4 → 0x4 → 0x4 → 0x4 → No event
- 0x4 → 0x0 → Ignition OFF event
- 0x0 → 0x4 → Ignition ON event

This prevents event spamming and ensures clean backend event logs.

3. Input Quality Handling

CmdIgnSts may momentarily oscillate due to:

- CAN noise,
- mechanical switch chatter inside the ignition system,
- transitions during engine cranking,

The firmware must therefore apply minimal debouncing, ensuring:

- false transitions (OFF→ON→OFF→ON within milliseconds) do not cause premature events;
- but responsiveness is not compromised.

Debounce limit must not exceed **100–200 ms**, ensuring:

- timely reaction to real ignition changes,
- suppression of jitter-induced false triggers.

This is important because overly aggressive filtering would cause late telemetry or trip transitions, whereas no filtering would produce unstable state behaviour.

4. CAN Dropout Conditions

If the firmware does not receive CmdIgnSts for a defined timeout period (recommended ~3 seconds):

- It must conservatively assume Engine OFF,
- Because the absence of ignition information typically indicates either ignition truly OFF, CAN bus inactive, or power state transitioning.

Firmware must then:

- Immediately halt telemetry scheduling,
 - Move Trip FSM toward stop evaluation,
 - Pause MIL evaluation and post-engine-off behaviours,
 - And log the dropout internally for debugging.
- This rule protects the device from continuing operational behaviour during vehicle shutdown or CAN bus inactivity.

4.1.3 MIL Status Monitoring (Engine Malfunction Indicator)

Purpose

MIL monitoring informs health alerts for engine malfunction or emission system errors.

Because MIL is time-sensitive and critical for diagnostics, firmware must follow strict timing and evaluation rules.

Mandatory Evaluation Logic

1. Delayed Evaluation Requirement: Firmware must *not* evaluate MIL immediately upon Engine ON. It must wait for 5 continuous seconds after detecting Engine ON to avoid transient MIL readings.
2. Alert Generation Rules:
 - MIL = 0x1 or 0x2 after 5s → MIL Alert ON
 - MIL = 0x0 after 5s → MIL Alert OFFAlerts must be generated only on state transitions, not continuously.
3. Telemetry Inclusion; MIL status must be included in periodic telemetry frames whenever Engine = ON.
4. Error Handling

If CAN ID 0x763 is missing for multiple cycles:

- The firmware must temporarily retain the last known MIL state.

- After a timeout, the MIL status must be marked unknown, and no new alerts must be generated until valid data is received.
5. MIL Reset Behaviour
- If MIL transitions from ON to OFF:
- Firmware must generate an Alert OFF notification.
 - Clearing of MIL must not reset trip states or telemetry schedules.

4.1.4 Handling Missing, Invalid, or Corrupted CAN Data

Purpose

The Malfunction Indicator Lamp (MIL) is one of the most important diagnostic signals exposed through the vehicle's CAN network. It reflects critical information related to emission system failures, engine operating anomalies, and regulatory compliance faults that must be communicated to backend systems and, in turn, to end users and maintenance personnel.

Because MIL transitions often indicate actionable service requirements, the firmware must monitor this signal with exceptional accuracy, ensuring that alerts are neither delayed nor triggered prematurely.

MIL monitoring is highly time-sensitive; it must consider engine warm-up behaviour, CAN stabilization periods after ignition, and system-wide transient conditions.

Therefore, firmware must enforce strict timing, evaluation logic, and error-handling rules to ensure reliable and trustworthy MIL alerting behaviour.

Processing Rules

1. Missing Data

If a required CAN signal is temporarily missing, the firmware must apply a controlled response strategy that prevents false transitions or abrupt behavioural changes:

- **Short Duration Absence (<3 seconds)**
When a CAN signal such as CmdIgnSts, VIN fragments, or MIL status is not received for a brief period, the firmware should continue using the last known valid value. This prevents unnecessary transitions, such as:
 - Engine ON to Engine OFF due to one missed frame
 - MIL ON/OFF oscillations
 - VIN validation aborts during transient gaps
Short-duration gaps can occur due to CAN arbitration, network load, or ECU processing delays.
- **Absence Beyond Timeout**
If the signal remains missing beyond the defined timeout window (typically 2–5 seconds depending on signal criticality), the firmware must assume a safe fallback

state.

Example:

- Missing CmdIgnSts → treat engine as OFF
 - Missing VIN → treat VIN as not acquired
 - Missing MIL → treat MIL state as unknown
- Safety defaults prevent stale or misleading CAN data from driving telemetry, events, or alert logic.

2. Corrupted Frames

Corrupted or malformed CAN frames must be rejected deterministically to maintain decoding integrity:

- **Invalid Length or Format**
Any frame whose data length, DLC, or structure does not conform to expected specifications must be discarded immediately.
- **Unexpected VIN_MSG Indexes**
Frames containing VIN_MSG values outside the expected sequence range must be treated as invalid.
The VIN assembly buffer must not accept fragments unless:
 - the VIN_MSG index is within expected bounds,
 - and the corresponding VIN_DATA payload conforms to format rules.
- **Malformed Payload Handling**
If payload bytes contain prohibited characters (non-ASCII, invalid VIN characters) or exhibit unexpected patterns, the frame must be ignored.

3. CAN Bus Down Scenario

When extended CAN silence or bus failure occurs, the firmware must detect and handle the situation gracefully:

- **CAN Inactivity Detection:** If no CAN frames at all are received for a prolonged duration, the firmware must classify the CAN subsystem as inactive.
This may occur due to ignition OFF, wiring issues, bus sleep, or ECU power-down.
- **LED CAN Indicator Reaction:** The CAN LED must reflect this condition by switching to the 2-blink “CAN not detected” pattern, providing clear field-level feedback.
- **Halt Telemetry Publishing:** Since most telematics data depends on CAN inputs, the firmware must pause all telemetry transmissions until CAN activity resumes.
This avoids transmitting stale or misleading vehicle data to backend systems.

- Resumption Logic : As soon as CAN frames restart, the firmware must immediately re-enable decoding, reevaluate engine state, and resume telemetry scheduling.

4. Event and Alert Dependencies

Many event and alert rules depend on valid CAN signals. Therefore:

- Events Must Not Trigger During CAN Failure: Events such as ignition ON/OFF, connection status, or VIN acquisition must not fire while CAN data is unavailable. This prevents false ignition transitions or connection status anomalies.
- MIL Evaluation Must Pause: The firmware must suspend MIL alert generation when CAN ID 0x763 is missing, ensuring that alerts are not generated based on stale MIL data.
- Trip FSM Must Not Advance: The device should not transition trips to PENDING or ACTIVE states without valid ignition or speed information.

4.1.5 CAN Subsystem Activation Rules

Table 4.1 — CAN Subsystem Status Across Device Lifecycle States

Lifecycle State	CAN Subsystem Status	Reason
Pre-Sales	Disabled	Device not in vehicle; no VIN or telematics allowed
Factory	Enabled	VIN acquisition required
Provisioned	Enabled	Engine state & MIL monitoring begin preparation
Authorized	Enabled	Subsystems must be ready for immediate activation
Customer	Enabled	Full telematics operation

4.2 LED Indication System

The LED indication system provides visual feedback regarding the operational status of key subsystems inside the OBD device. Each LED reflects the health and working condition of

the Power, GPS, GSM, and CAN interfaces. The firmware must implement the LED behaviours exactly as defined in this specification.

4.2.1 Power LED (Monochrome)

Purpose

Indicates that the device is powered and operational.

Table 4.2 — LED Status and Blink Patterns for Authorization State Update

LED ON	LED Blink Pattern	Authorize State Update pattern at VPC to identify the success
Yes	No Blink	Device is powered ON and firmware is running

- Power LED must never blink.
- Power LED has no role in lifecycle or telematics states; it simply reflects power integrity.

4.2.2 GPS LED (Blue)

Purpose

Indicates GNSS subsystem status, satellite visibility, and the device's ability to compute location.

Table 4.3 — LED Indicators for GNSS Working Condition Status

Working Condition Indication	LED ON	LED Blink Pattern	Authorize State Update pattern at VPC to identify the success
Yes	–	Blink 1 time with 2-second delay	Location update happening; satellite signal is strong
No	–	Blink 2 times with 3-second delay	Searching for location update; insufficient or unstable GPS signal

- GPS LED never uses a steady ON state.
- 1-blink indicates usable GNSS fix.
- 2-blink indicates searching mode (indoors, tunnels, poor sky view).

4.2.3 GSM LED (Green)

Purpose

Indicates cellular modem registration status and network readiness for data transfer.

Table 4.4 — LED Indicators for Mobile Network Working Condition Status

Working Condition	LED ON	LED Blink Pattern	Authorize State Update pattern at VPC to identify the success
Yes	–	Blink 1 time with 2-second delay	Mobile network signal strength is good for sending/receiving data
No	–	Blink 2 times with 3-second delay	Searching operator; attempting to latch to mobile network

- 1-blink pattern reflects successful network registration.
- 2-blink pattern reflects network search / weak signal / SIM/operator instability.

4.2.4 CAN LED (Red)

Purpose

Indicates the health of the CAN communication interface and the availability of OBD data.

Table 4.5 — LED Indicators for CAN Communication Working Condition Status

Working Condition	LED ON	LED Blink Pattern	Authorize State Update pattern at VPC to identify the success
Yes	–	Blink 1 time with 2-second delay	CAN communication is active; device is receiving OBD data
No	–	Blink 2 times with 3-second delay	CAN disconnected; RX buffer is null

- 1-blink pattern confirms healthy CAN activity.
- 2-blink indicates no CAN frames, often due to ignition OFF, wiring issues, or inactive vehicle systems.

4.3 Telemetry Publishing Logic

The firmware is responsible for publishing vehicle telemetry data to the CVIP backend based on well-defined engine state transitions and timing rules.

Telemetry publishing follows a strict schedule that is tied directly to the engine ON/OFF condition, ensuring that the backend receives periodic status updates during active vehicle operation and a final set of indicator packets when the engine shuts down.

This section defines all firmware behaviours associated with telemetry timing, data sampling, and conditional publishing.

4.3.1 Purpose

Telemetry publishing provides real-time visibility into vehicle operation, enabling backend systems to compute driving behaviour, trip summaries, alerts, and diagnostics.

Because continuous transmission is not required when the engine is off, and because OEM backend systems must receive one or two indicator samples immediately after shutdown, firmware must implement a precise and predictable telemetry schedule.

The telemetry engine must:

- publish only during engine ON,
- follow a 60-second publish cycle,
- send two additional samples just after engine OFF,
- then remain silent until the next valid engine ON event.

These rules prevent unnecessary network usage and ensure backend analytics remain accurate.

4.3.2 Telemetry Trigger Conditions

Telemetry is triggered exclusively by engine state, derived from CmdIgnSts:

- Engine ON → Telemetry active
- Engine OFF → Telemetry halted, except for shutdown indicator packets

No other signal (speed, movement, trip status, CAN flow) can independently trigger telemetry publishing.

4.3.3 Telemetry Publishing Rules During Engine ON

When the engine is ON, the firmware must:

1. Publish telemetry every 60 seconds without deviation.

2. Ensure that the first telemetry sample is taken at the moment engine transitions to ON.
3. Ensure that the timing for consecutive packets follows a strict **T, 2T, 3T...** cadence, where **T = 60 seconds**.
4. Continue publishing until engine transitions to OFF.

Table 4.6 — Telemetry Timing Window (Engine ON)

Time	Engine State	Publish?	Packet ID
T	ON	Yes	D1
2T	ON	Yes	D2
3T	ON	Yes	D3
4T	ON	Yes	D4

The firmware must ensure:

- No drift greater than a few seconds per interval,
- No skipping of intervals unless CAN data is invalid,
- No publishing when engine state is unstable.

4.3.4 Telemetry Behaviour During Engine OFF Transition

When CmdIgnSts transitions from ON → OFF, firmware must:

1. Immediately stop the periodic 60-second telemetry timer.
2. Trigger two indicator telemetry packets:

D5: First post-engine-off packet (Shutdown Indicator 1)

D6: Second post-engine-off packet (Shutdown Indicator 2)

These packets must be sent even if CAN data becomes unavailable immediately after engine shutdown. After D5 and D6 are published, telemetry publishing must stop completely until the engine is ON again.

Table 4.7 — Timeline for Engine OFF Period

Time	Engine State	Publish?	Packet
4.5T	OFF	Yes	D5
5T	OFF	Yes	D6
6T	OFF	No	–
7T	OFF	No	–

4.3.5 Telemetry Suspension Rules

The firmware must suspend telemetry publishing under the following conditions:

1. Engine OFF (after sending D5 and D6).
2. CAN communication inactive for more than the defined timeout.
3. GSM connectivity unavailable, preventing secure data upload.
4. Device not in Customer state (telemetry only permitted in Customer lifecycle).

Suspension ensures the backend receives telemetry only during valid drive cycles.

4.3.6 Telemetry Resumption Rules

Telemetry must resume only when:

- Engine transitions from OFF → ON,
- CAN becomes active and CmdIgnSts is valid,
- GSM network is latched and data upload is permitted.

Upon resumption:

- Firmware must restart the T = 60s timer from zero.
- Firmware must publish the first ON-state sample immediately at the next scheduled time slot.

4.3.7 Error Handling in Telemetry Publishing

1. Missing CAN Data
 - If CAN frames are missing during engine ON, firmware may skip the scheduled telemetry interval to avoid sending invalid data.
2. Weak GSM Signal
 - If GSM LED indicates “searching,” telemetry must queue until network resumes.
3. Duplicate Engine OFF Events
 - If Engine OFF is triggered repeatedly due to noisy transitions, firmware must send D5/D6 only once per shutdown.
4. Timestamp Validity
 - If GPS time unavailable, fallback to modem time or RTC.

4.3.8 Telemetry Constraints and Requirements

- Telemetry schedule accuracy: ± 2 seconds.
- Minimum interval: 60 seconds (no faster periodic publishing allowed).
- No telemetry before Customer state.
- No telemetry during VIN acquisition or provisioning.
- Telemetry packets must be retried on network failure.

4.4 Event Framework

This subsection defines the event generation criteria for **Connection Status** and **Ignition Status** as part of the standardized event framework. All events follow an **on-occurrence** triggering model and are published to the CVIP endpoint using:

/dongle/{vinnum}/{protocol}{dataformat}/events

Connection Status Event

- Event is generated whenever the dongle’s network connectivity state changes, following the on-occurrence trigger defined in the Event Framework.
- Since this parameter is device-specific and not derived from CAN, the connectivity evaluation logic resides in Traxo’s internal implementation layer.

Ignition Status Event

- Ignition state is determined from CAN ID 0x46C / BH2-CAN / CmdIgnSts, where values 0x04 (RUN) and 0x05 (START) represent IGNITION ON, and any other value represents IGNITION OFF.
- An event is published whenever the interpreted ignition state transitions between OFF → ON or ON → OFF.

WIP Note:

Traxo-owned fields such as Connection Status logic definition and Dongle handling details remain WIP and will be finalized during integration.

4.5 Alert Framework

The Alert Framework defines all high-priority conditions under which the firmware must generate and transmit alerts to the CVIP backend.

These alerts represent **exceptional or safety-relevant situations**, such as engine malfunction indications, abnormal vehicle movement, or aggressive driving patterns, and therefore require immediate and accurate reporting.

Unlike periodic telemetry, which follows a fixed schedule, alerts are evaluated continuously based on signal changes and are triggered when specific logical conditions become true or false.

The firmware must implement alert generation in such a way that:

- each alert is raised only when a real transition into an abnormal condition occurs,
- each alert is cleared only when the corresponding condition returns to normal,
- no duplicate alerts are generated while a condition remains continuously active, and
- alert evaluation is suspended when the underlying sensor or CAN information is unavailable.

The subsections below define the alert types, their signal sources, evaluation logic, and reporting behaviour.

4.5.1 MIL Alert (*Malfunction Indicator Lamp*)

The MIL alert indicates that the vehicle's engine or emission system has reported a diagnostic condition requiring attention.

This alert is based on the MIL status reported by the engine control system over the CAN bus and is tightly coupled with engine state and regulatory expectations.

Signal Source

- CAN ID: 0x763

- Signal: ENGINE_MIL_STAT

Evaluation Timing

The firmware must not evaluate or act on MIL information immediately at engine start. After the engine transitions to ON, the firmware must wait for a continuous period of five seconds before evaluating the MIL status, allowing all ECUs to complete their internal initialization and avoiding false indications during startup.

Alert Conditions

- The firmware must generate a MIL Alert ON when:
 - the engine has been in the ON state for at least five seconds, and
 - ENGINE_MIL_STAT indicates a non-normal condition (for example, values corresponding to “On” or “Blinking” as defined in the CAN signal specification).
- The firmware must generate a MIL Alert OFF when:
 - the engine has been in the ON state for at least five seconds, and
 - ENGINE_MIL_STAT returns to the value representing “Off”.

Operational Behaviour

Once a MIL Alert ON has been generated, the firmware must not repeatedly reissue the same alert as long as the MIL status remains continuously in the abnormal range. Similarly, once a MIL Alert OFF has been sent, no further MIL-clear alerts must be sent until the MIL again transitions from Off back to an abnormal condition. If CAN frames for ENGINE_MIL_STAT are not received for a defined timeout period, the firmware must treat the MIL status as unknown and must refrain from generating any new MIL ON or OFF alerts until valid data is again available.

4.5.2 Device Removal Alert

The Device Removal alert indicates that the OBD dongle has likely been unplugged or disconnected from the vehicle OBD-II port in an abnormal or unexpected manner.

Trigger Conditions

The firmware must infer device removal when it observes a combination of:

- sudden loss of vehicle power or OBD supply, and
- abrupt loss of CAN activity, while
- the device may still briefly remain powered through any internal energy storage (if present), allowing the alert to be transmitted.

Operational Behaviour

When these conditions are detected in a pattern that clearly indicates disconnection rather than normal ignition OFF behaviour, the firmware must generate a Device Removal Alert ON. This alert must be raised only once per removal incident. The corresponding Device Removal Alert OFF must be generated only after the device is reconnected, CAN communication is restored, and the system has confirmed stable vehicle-side power and bus activity.

4.5.3 Speed Limit Alert

The Speed Limit alert indicates that the vehicle is travelling at a speed higher than a configured limit.

Signal Source

- Vehicle speed signal obtained from CAN as per the BH2-CAN mapping.

Alert Conditions

- The firmware must generate a Speed Limit Alert ON when the measured vehicle speed is greater than the configured speed limit for a continuous period that exceeds a minimum evaluation window. This evaluation window is intended to prevent alerts from being raised for very brief or transient overshoots.
- Once the vehicle speed falls back below the configured limit and remains below that limit for a stable interval, the firmware must generate a Speed Limit Alert OFF.

Operational Behaviour

The firmware must ensure that the speed threshold and time window are configurable (from cloud or preset values) and that repeated fluctuations around the threshold do not cause rapid toggling between alert ON and alert OFF states.

4.5.4 Engine Idle Alert

The Engine Idle alert indicates that the vehicle engine has been running for an extended duration while the vehicle remains stationary.

Signal Sources

- Engine state derived from CmdIgnSts (Engine ON), and
- Vehicle speed equal to zero for a sustained duration.

Alert Conditions

- The firmware must generate an Engine Idle Alert ON when the engine is ON and the vehicle speed remains at zero for longer than a configured idle timeout period.

- The firmware must generate an Engine Idle Alert OFF when either the vehicle begins moving (speed becomes greater than zero) or the engine transitions to the OFF state.

Operational Behaviour

This logic must be implemented to distinguish between normal short idling (for example, traffic signals) and prolonged idle behaviour by using appropriate timing thresholds. Alerts must not be reissued repeatedly if the engine remains continuously in the idle state.

4.5.5 Towing Alert

The Towing alert indicates that the vehicle is moving while the engine is OFF, suggesting that the vehicle may be being towed or moved without ignition.

Signal Sources

- Engine state indicating OFF, and
- Movement detected either through vehicle speed from CAN or from an onboard motion sensor such as an accelerometer.

Alert Conditions

- The firmware must generate a Towing Alert ON when the engine is OFF and the motion parameters (speed or accelerometer readings) indicate sustained movement beyond a configured movement threshold.
- When this abnormal movement ceases—because the vehicle stops or the engine transitions to ON—the firmware must generate a Towing Alert OFF.

Operational Behaviour

The firmware must apply suitable thresholds and timing so that minor vibrations, loading operations, or small disturbances do not produce towing alerts. Only actual, sustained movement with engine OFF should qualify.

4.5.6 Parking Disturbance Alert

The Parking Disturbance alert indicates that the parked, engine-OFF vehicle experiences a physical disturbance, such as an impact or strong vibration.

Signal Source

- Accelerometer or other motion-sensing hardware within the device.

Alert Conditions

- The firmware must generate a Parking Disturbance Alert ON when the engine is OFF and the measured acceleration or vibration exceeds a configured disturbance threshold, indicating non-trivial physical interaction with the vehicle.
- The firmware must generate a Parking Disturbance Alert OFF once the sensed motion returns to baseline and remains stable for a defined settling period.

Operational Behaviour

The detection algorithm must be chosen such that small background vibrations (for example, due to wind or someone closing a nearby door) do not generate false alerts, while genuine impacts, pushing, or jolting of the vehicle are reliably detected.

4.5.7 Harsh Acceleration Alert

The Harsh Acceleration alert captures aggressive throttle behaviour that is beyond normal driving patterns.

Signal Source

- Longitudinal acceleration from an accelerometer, and/or
- Rate of change of vehicle speed derived from CAN.

Alert Conditions

- The firmware must generate a Harsh Acceleration Alert ON when the measured acceleration or speed increase exceeds a configured harsh-acceleration threshold over a short evaluation interval.
- Once acceleration values return to within normal bounds, the firmware must generate a Harsh Acceleration Alert OFF.

Operational Behaviour

The thresholds must be selected such that normal acceleration, lane changes, and merges do not trigger the alert, while strong and sudden accelerations clearly do.

The firmware must also avoid repeated toggling of alerts if the driver maintains a sustained but acceptable acceleration profile.

4.5.8 Harsh Braking Alert

The Harsh Braking alert indicates sudden and significant braking behaviour that may correspond to risky driving episodes or near-miss events.

Signal Source

- Longitudinal deceleration from accelerometer data, and/or
- Rapid negative change in vehicle speed from CAN.

Alert Conditions

- The firmware must generate a Harsh Braking Alert ON when the deceleration magnitude exceeds a configured harsh-braking threshold during a short interval.
- Once deceleration levels fall back into the normal range, the firmware must generate a Harsh Braking Alert OFF.

Operational Behaviour

The implementation must ensure that ordinary deceleration during normal braking does not generate alerts.

Only genuinely abrupt and strong braking actions should cause harsh braking alerts, with appropriate filtering to prevent chattering.

4.5.9 Sharp Turn Alert

The Sharp Turn alert indicates high lateral forces experienced during cornering, which may be associated with aggressive or unsafe turning behaviour.

Signal Source

- Lateral acceleration from an accelerometer, and/or
- Gyroscope or similar orientation sensor, if available.

Alert Conditions

- The firmware must generate a Sharp Turn Alert ON when the measured lateral acceleration or turn rate exceeds a configured sharp-turn threshold while the vehicle is in motion.
- Once the lateral forces subside and values fall below the threshold for a stable duration, the firmware must generate a Sharp Turn Alert OFF.

Operational Behaviour

The threshold must be configured so that normal lane changes, gentle curves, and gradual turns do not produce alerts. The firmware must detect only genuinely sharp or abrupt turning manoeuvres.

4.5.10 Alert Reporting Requirements

Alert reporting must follow a strict set of rules to ensure timely and reliable delivery:

- Alerts must be sent to the backend as soon as reasonably possible after their ON or OFF conditions are detected, and must not be deferred until the next periodic telemetry interval.
- For every alert type, the firmware must generate exactly one Alert ON message at the moment the condition first becomes true, and exactly one Alert OFF message when the condition ceases, without repeating the same state multiple times while conditions remain unchanged.
- Each alert message must include sufficient context, such as timestamp, VIN, device identity (for example, MSISDN), and the relevant measured parameters that caused the alert, so that backend systems can interpret and act on the alert correctly.
- If CAN data or other required sensor inputs are unavailable—for example, during CAN bus inactivity—the firmware must temporarily suspend evaluation of alerts that depend on those signals and must not generate any new alerts based on incomplete or stale information.
- If GSM connectivity is unavailable at the moment an alert needs to be sent, the firmware must queue the alert in local storage, mark it for retry, and transmit it when network connectivity is restored, preserving the original event time.
- No alert generation or transmission must occur while the device is still in lifecycle states before Customer activation; alert logic becomes fully active only once the device is in the Customer state and operational telematics have begun.

4.6 Trip Management

The Trip Management subsystem defines how the firmware interprets ignition states, vehicle motion, and distance accumulation to classify driving activity into well-defined trip phases.

Trips are fundamental to backend analytics such as drive scoring, distance computation, fuel estimation, and behavioural insights.

To ensure consistency in trip boundaries, the Trip Engine must use strict thresholds and timing windows as defined by OEM parameters and must avoid false trip starts or early trip terminations.

The Trip Engine operates as a finite state machine (FSM) consisting of three primary states:

- TRIP_IDLE
- TRIP_PENDING
- TRIP_ACTIVE

Transitions between these states are driven exclusively by ignition status, vehicle movement, configurable thresholds, and evaluation timers.

4.6.1 Trip FSM Overview

Table 4.8 — Trip State Definitions and Indications

Trip State	Indication
TRIP_IDLE	Vehicle not in an active trip; engine OFF or insufficient movement.
TRIP_PENDING	Engine ON or vehicle moving, but minimum trip conditions not yet met.
TRIP_ACTIVE	All requirements met; trip officially started and telemetry considered part of a valid drive.

The Trip Engine ensures that a trip begins only when meaningful movement occurs and ends only after the engine remains OFF long enough to guarantee that the drive is complete.

4.6.2 TRIP_IDLE State

Entry Conditions

- Engine is OFF, *or*
- Engine has turned ON but movement has not yet satisfied the criteria to enter PENDING state, *or*
- A previous trip has ended and ignition remains OFF beyond the configured MIN_IGN_OFF_TIME.

Operational State

- Distance accumulation is not counted toward any trip.
- Telemetry may still be generated (Engine OFF indicator packets), but trip fields remain inactive.
- All trip timers and counters must be reset.
- Engine OFF transition does **not** immediately end any trip; termination happens only after OFF-time exceeds the configured threshold.

Exit Condition

Transition to **TRIP_PENDING** occurs when:

- Engine transitions to ON, or
- Movement is detected (speed > 0 or accelerometer motion), even if trip criteria are not yet fully satisfied.

4.6.3 TRIP_PENDING State

Represents the intermediate stage where the engine is ON and movement may be beginning, but minimum thresholds required to declare a trip have not yet been met. This prevents false trip creation during short idling events, minor movements, or brief ignition cycles.

Entry Condition

Transition from TRIP_IDLE occurs when:

- Engine ON is detected, or
- Vehicle shows initial motion sufficient to indicate possible drive initiation.

Operational State

While in TRIP_PENDING:

- Firmware must accumulate distance incrementally.
- Firmware must measure continuous engine ON duration.
- No trip is reported to backend yet; trip metadata is internal only.
- Movement is monitored to determine whether the vehicle is genuinely beginning a drive or simply idling.
- If the vehicle remains stationary for too long, the PENDING state may remain without transitioning to ACTIVE.
- All thresholds are evaluated continuously.

Trip Start Criteria

Transition from PENDING to ACTIVE occurs only when *all* of the following are satisfied:

1. MIN_TRIP_DISTANCE has been crossed.
2. Engine status remains ON (no OFF-to-ON flicker).
3. CAN data remains valid without missing-speed intervals.

Failure to Start Condition

If engine transitions to OFF before meeting MIN_TRIP_DISTANCE, the FSM returns to TRIP_IDLE without creating or reporting a trip. This prevents zero-distance or micro-movement trips.

4.6.4 TRIP_ACTIVE State

Represents a valid, ongoing trip.

All telemetry samples generated in this phase must be tagged as part of the same trip for backend correlation.

Entry Condition

Trip begins when:

- Minimum distance threshold is exceeded while in TRIP_PENDING.

Operational State

While in TRIP_ACTIVE:

- Firmware must continuously accumulate distance, time, and any trip-related metrics.
- All telemetry is considered part of an active trip and must contain trip identifiers.
- Alerts such as harsh acceleration, harsh braking, and sharp turns become relevant and must be evaluated.
- The trip continues as long as engine remains ON or movement persists.

The trip must not end abruptly during brief engine OFF-on flickers, CAN dropouts, or intermittent signal loss.

Exit Condition

Transition to TRIP_IDLE occurs only when:

1. Engine transitions to OFF, and
2. Engine remains OFF longer than MIN_IGN_OFF_TIME, ensuring genuine trip termination, and
3. No significant movement is detected during the OFF window.

Post-Trip Actions

Upon confirming trip end:

- Firmware must finalize trip distance and timing metrics.
- The last two telemetry packets (D5, D6) may be part of shutdown reporting.

- Trip summary fields must be reset only after end-of-trip metadata is committed.

4.6.5 Trip Distance and Timing Thresholds

The following configuration parameters govern trip behaviour:

MIN_TRIP_DISTANCE

- Minimum distance required to consider a movement as a valid trip.
- Prevents false or zero-distance trips.

MIN_IGN_OFF_TIME

- Minimum duration engine must remain OFF before a trip is officially considered ended.
- Avoids early trip termination during short stops (fuel pump stoppage, traffic, etc.).

4.6.8 Functional Constraints

- No trip evaluation must occur before Customer lifecycle state.
- VIN must be successfully decoded prior to trip logic activation.
- Trip computations must not conflict with telemetry suspension rules.
- Trip FSM must be deterministic and restart gracefully after reboot.

5. Southbound Communication Requirements

The Southbound Communication subsystem defines how the OBD firmware interacts with the CVIP backend across all lifecycle stages.

This section specifies the API endpoints, identities (IMEI, VIN, MSISDN), payload responsibilities, and behavioural constraints that govern communication between the device and the cloud.

Each lifecycle state defined in Section 3 mandates a specific cloud exchange pattern, and the firmware must follow these interactions exactly to maintain pairing integrity, provisioning correctness, and secure end-to-end telematics flow. A concise summary of all required API calls and their usage is provided in Table 5-1, followed by telemetry and alert transmission rules in Table 5-2 and Table 5-3.

5.1 Overview of Southbound Lifecycle Communication

During the device lifecycle, the firmware transitions through Pre-Sales, Factory, Provisioned, Authorized, and Customer states.

Each transition requires a specific southbound action, either to request a certificate, join the platform, acknowledge commands, or publish operational data.

The firmware must ensure that API calls occur only when the device is in the correct lifecycle state, using the identity dictated by that state.

Retry behaviour, identity selection, and payload formation must all comply with the associated API requirements and security expectations. The complete set of required interactions is summarized in Table 5-1.

5.2 Telemetry Publishing Rules (Southbound Layer)

Once the device reaches Customer state (refer Section 3.2.5), telemetry becomes the primary data channel used to communicate operational vehicle information.

Telemetry is published strictly based on engine ON/OFF transitions, and the firmware must observe the timing and suspension rules described in Section 4.3.

The southbound layer must ensure reliable packet delivery, queueing if GSM connectivity is unavailable, and retry logic for cloud timeouts. Telemetry behaviour during ON and OFF windows is summarized in Table 5-2.

Table 5-1 — Southbound API Interaction Summary

Lifecycle State	API Endpoint	Identity Used	Purpose	Firmware Behaviour
Pre-Sales	/dongle/{IMEI}/{protocol}{dataformat}/csrAccessTokenReq	IMEI	Request CSR access token for certificate onboarding	Send once firmware is flashed; store token; no telemetry allowed
Factory	<i>No API interaction</i>	IMEI	VIN acquisition only	Decode VIN from CAN; remain in Factory until VIN is complete
Provisioned	/dongle/{vinnum}/{protocol}{dataformat}/deviceJoined	VIN	Register VIN-bound device with CVIP	Send during entry; wait for backend acknowledgment
Authorized	/dongle/vinnumber/{protocol}{dataformat}/commandResponse	VIN	Exchange authorization commands and responses with backend	Poll/listen for cloud commands; telemetry disabled
Customer	/commandResponse, /telemetryUpload, /eventUpload, /alertUpload	MSISDN	Full telematics, events, alerts, and command flow	Publish telemetry/events/alerts and handle cloud commands

Table 5-2 — Telemetry Transmission Rules

Condition	Telemetry Behaviour
Engine ON	Publish every 60 seconds
Engine OFF	Publish D5 and D6 shutdown packets only, then stop
CAN inactive	Suspend telemetry until CAN resumes
GSM unavailable	Queue packet and retry on network recovery
Not in Customer state	Telemetry disabled

5.3 Event and Alert Transmission Requirements

Events and alerts represent condition-driven notifications that must be transmitted to CVIP outside the regular telemetry schedule.

Unlike periodic telemetry, events and alerts must be delivered immediately upon occurrence, provided GSM connectivity is available.

If connectivity is temporarily unavailable, the firmware must queue these messages and transmit them at the earliest opportunity once the network is restored.

The firmware must ensure that event and alert messages reflect state transitions only, in order to avoid duplicate reporting. A summary of event and alert upload mappings is shown in Table 5-3.

Table 5-3 — Event & Alert Upload Summary

Trigger Type	Upload Behaviour	Upload Endpoint
Ignition ON/OFF	Upload immediately on state change	/eventUpload
MIL Alert ON/OFF	Upload immediately on transition	/alertUpload
Driving behaviour alerts (harsh acceleration/braking/turn), towing, parking disturbance	Upload on occurrence	/eventUpload

6. Annexures

The following annexures are included:

- **Annexure A — CAN Signal Definitions**
- **Annexure B — Trip State Machine Diagram**
- **Annexure C — Configuration Parameters & Thresholds**

Annexure A — CAN Signal Definitions

This annexure consolidates all CAN-related references used in OBD-II decoding, ignition detection, diagnostic evaluation, and trip computation. It serves as the definitive reference for firmware handling of CAN signals.

A.1 VIN Extraction Signals (CAN ID: 0x3E0)

Field Name	Description	Data Format	Notes
VIN_MSG	Fragment index of VIN sequence	1 byte	Determines position of VIN_DATA in final 17-byte VIN
VIN_DATA	VIN fragment (up to 7 bytes)	7 bytes	Reassembled in strict VIN_MSG order

- VIN must be exactly 17 alphanumeric characters.
- Fragment arrival may be out of sequence; firmware must reorder correctly.

A.2 Ignition State Signal (CAN ID: 0x46C — CmdIgnSts)

Value	Interpretation	Engine State
0x4	RUN	Engine ON
0x5	START	Engine ON
Any other value	ACC/OFF/Unknown	Engine OFF

Notes:

- This is the **exclusive** source of engine state.
- Trip FSM, telemetry scheduling, and MIL evaluation depend entirely on this mapping.

A.3 MIL Status Signal (CAN ID: 0x763 — ENGINE_MIL_STAT)

Value	Meaning
0x0	MIL OFF
0x1	MIL ON
0x2	MIL Blinking

- - MIL evaluation begins after **5 seconds** of stable Engine ON.
 - Used to generate MIL ON/OFF alerts.

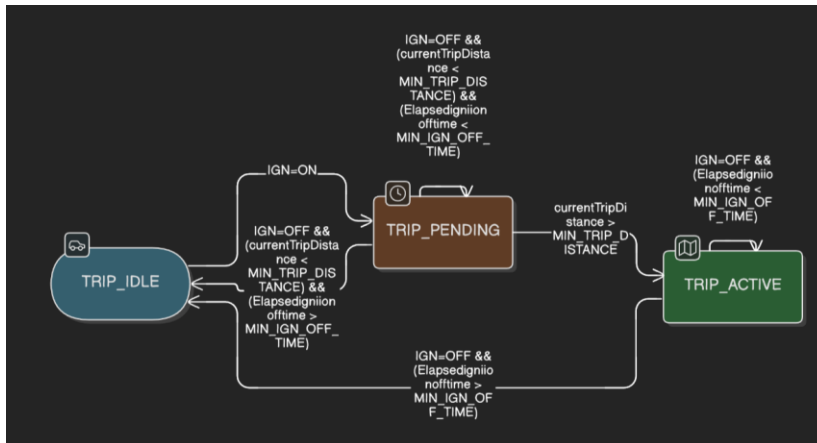
Annexure B — Trip State Machine Diagram

This annexure contains the Trip Finite State Machine (FSM) used by the firmware to classify vehicle operation into TRIP_IDLE, TRIP_PENDING, and TRIP_ACTIVE states.

The diagram illustrates all possible transitions between states and the associated entry/exit conditions governed by engine state, vehicle movement, and configurable trip thresholds.

The Trip FSM serves as the authoritative reference for the requirements defined in Section 4.6 (Trip Management).

B.1 Trip FSM Diagram



Annexure C — Configuration Parameters and Thresholds

This annexure consolidates all configurable parameters referenced throughout the firmware requirements.

These parameters influence trip evaluation, telemetry timing, alert generation, and system reliability.

Values may be provisioned by CVIP, updated over-the-air, or embedded as firmware defaults based on OEM requirements.

The tables below serve as the authoritative reference for configuration items mentioned in **Sections 4.1 through 4.6**.

C.1 Trip Engine Parameters

Parameter	Description	Usage in Firmware
MIN_TRIP_DISTANCE	Minimum distance required to classify a valid trip (meters).	Governs transition from TRIP_PENDING → TRIP_ACTIVE.
MIN_IGN_OFF_TIME	Minimum duration (seconds) engine must remain OFF to end a trip.	Governs transition from TRIP_ACTIVE → TRIP_IDLE.
IDLE_TIMEOUT_THRESHOLD	Maximum duration Engine ON + speed=0 is allowed.	Used to trigger Engine Idle Alert.

C.2 Telemetry Parameters

Parameter	Description	Usage
CFG_TELEMETRY_INTERVAL_SEC	Telemetry publish interval when Engine ON (default: 60 seconds).	Controls periodic telemetry D1–D4
CFG_SHUTDOWN_SAMPLE_COUNT	Total post-engine-off telemetry samples (value = 2).	D5 and D6 packets
TH_CAN_INACTIVITY_TIMEOUT_MS	Timeout beyond which CAN is considered inactive.	Telemetry suspension + Trip FSM pause

CFG_GSM_RETRY_INTERVAL_SEC	Retry interval for queued packets when GSM signal is weak/unavailable.	Telemetry/event/alert retry
-----------------------------------	--	-----------------------------

C.3 Alert Threshold Parameters

Parameter	Description	Usage
TH_SPEED_LIMIT_KMH	Maximum permitted speed; exceeding triggers Speed Limit Alert.	Speed limit alert logic
TH_HARSH_ACCELERATION_MPS2	Acceleration threshold indicating harsh acceleration.	Harsh Acceleration Alert
TH_HARSH_BRAKING_MPS2	Deceleration threshold indicating harsh braking.	Harsh Braking Alert
TH_LATERAL_G_FORCE_MPS2	Lateral acceleration (G-force) threshold for sharp turn detection.	Sharp Turn Alert
TH_PARKING_DISTURBANCE_G	IMU sensitivity threshold for parking disturbance alert.	Parking Disturbance Alert

C.4 MIL Evaluation Parameters

Parameter	Description
CFG_MIL_EVAL_DELAY_SEC	Required stable Engine ON duration before MIL evaluation (5 seconds).

C.5 Retry, Recovery, and Timeout Parameters

Parameter	Description
CFG_API_RETRY_LIMIT	Maximum number of retry attempts for cloud API calls.
CFG_API_BACKOFF_MULTIPLIER	Exponential backoff multiplier for retry scheduling.
CFG_GSM_MAX_QUEUE_SIZE	Maximum number of buffered messages during GSM outage.
CFG_TIMESTAMP_FALLBACK_SRC	Fallback time source when GNSS time is unavailable (RTC/modem).

7. References

1. **TE 01_PCA Motors_M6 Jeep Migration_SouthBound_Interface_Specification_V1.0.pdf**
 - Source for device lifecycle states, onboarding flows, API endpoints, and VIN provisioning rules.
2. **TSRS Inputs from STLA.xlsx**
 - Source for telemetry timing definitions, event/alert logic, LED behaviour, and CAN signal mappings.
3. **BH2-CAN Signal Documentation**
 - Source for CmdIgnSts (0x46C), VIN frames (0x3E0), MIL status (0x763), and associated decoding rules.